

Tab 1

LAB REPORT

SOEN-341

Software Process

Lab Section: SP-X

Sprint 1

CareerConnect - Job Search and Application Tracking Platform

Philip	Ear	40334331
Vincent	Syvrais	40343752
Wyndair Daryl	Bartolome	40310244
Benjamin	Senay	40341705
Xavier	Lagace	40341466
Salvatore	Fulminis	40210805

Lab Instructor: Aya Al Haj

Date Due: September 28th , 2026

“We certify that this submission is our original work and
meets the Faculty’s Expectation of Originality.”

README File:

CareerConnect

A web-based platform designed to help job seekers manage their job search activities.

Git & Workflow Guide

To keep our repository organized and avoid breaking code, we follow a specific branch strategy. Please use this guide whenever you are working on features.

Important Rules

1. **Please do not try pushing directly to `main`.** Your local code must enter the `dev` branch first.
2. **`dev` is our default branch.** Please only push to `dev`. `main` is the **fully functional, holy-grail** codebase.

Now, to push `dev` code to `main`, create a **Pull Request** (PR). Someone else than yourself will review it and decide what to do with it. PRs may become mandatory for `dev` in the future, but for simplicity and learning, only the `main` branch has these rules.

How I'd start coding

1. Grab a Task

Go to the Github Project Board, the "Projects" tab on the top bar. Assign an **Issue** in the **Backlog** column to yourself and drag it to **In Progress**.

2. Update Local Dev Code

Before you write anything, make sure you pull the branch to ensure it's up-to-date on your machine.

```
``bash
git checkout dev
git pull origin dev
``
```

3. Save Your Work (Commit)

After making changes, add and commit your updates:

```
``bash
git add .
git commit -m "Briefly explain what you changed or built"
``
```

4. Push Your Code

Once you've done that, you can push the changes to the `dev` branch.

```
``bash
git push origin dev
``
```

It's not working

If Git blocks you from pushing, then someone may have pushed right before you. So run:

```
```bash
git pull origin dev
```
```

Resolve the conflicting lines in your IDE (I use VSCode), finalize your version and then finish the push with:

```
```bash
git add .
git commit -m "Fix merge conflict"
git push origin dev
```
```

Running the Server

Navigate to the server directory:

```
```bash

cd server
```

```
```
```

Install the project dependencies:

```
```bash

npm ci
```

```
```
```

Install Multer for resume file uploads:

```
```bash

npm install multer
```

```
```
```

Run the development server:

```
```bash

npm run dev
```

```
```
```

Running the Client

The CareerConnect frontend is built using React and Vite.

Navigate to the client directory:

```
```bash
```

```
cd client
```
```

Install the project dependencies:

```
```bash
npm ci
```
```

Run the development client:

```
```bash
npm run dev
```
```

The frontend should run on:

```
```text
http://localhost:5173
```
```

Open this address in your browser to access CareerConnect.

Make sure the backend server is also running on:

```
```text
http://localhost:4000
```
```

For the complete application to work, both the client and server should be running at the same time.

GitHub Repository Link: <https://github.com/Strykerxman/CareerConnect.git>

Task Separation and Role for each member

Member A: Scrum master (Xavier)

Repository setup (#11)

- Create the GitHub repo and add all five teammates as collaborators, plus the TA/instructor if the repo is private.
- Create a `develop` branch. Protect `main` and `develop`: require a PR, one approval, and passing CI, and block direct pushes.
- Create the GitHub Project board with columns Backlog → Ready → In Progress → In Review → Done.
- Run `bash scripts/create-issues.sh`, then add all issues to the board and assign each one to its owner from this list.
- Commit the folder skeleton: `docs/`, `meeting-minutes/`, `AI_Log/<each name>/`, and `docs/sprint-1/`. Git ignores empty folders, so put a `.gitkeep` in each.

Team process (#13)

- Take `docs/process.md` to the first meeting and get the team to agree on or change each rule: branching, PR rules, review turnaround, DoR, DoD.
- Update the file with whatever you decide. The TA may ask why you chose these rules, so be ready to explain.

Sprint planning (#14)

- Replace Member A–F with real names and each person's real weekly hours in `docs/sprint-1/sprint-plan.md`.
- Run a quick planning-poker round on the story points. Adjust the numbers if the team disagrees; that's evidence the estimates are yours.
- Keep Appendix A in sync with GitHub: issue numbers, owners, dates, and status. Update the status column the day before each TA meeting.

Meeting minutes (#17)

- Copy `meeting-minutes/TEMPLATE.md` for every meeting, named like `2026-09-29-planning.md`.
- Record attendance, decisions, action items linked to issues, and any AI use during the meeting.
- Commit the minutes within a day of each meeting.
- You need at least a planning meeting, the brainstorm, and each weekly sync.

Member B: Backend, authentication (Salvatore)

Database schema (#19)

- Own `server/src/db.js`: the `users` and `profiles` tables.
- Understand the design choices well enough to explain them:
 - email is unique and case-insensitive
 - role is restricted to `job_seeker` or `recruiter`
 - a profile is deleted along with its user

Registration endpoint (#20)

- Own `POST /api/auth/register` in `routes/auth.js` and `validation.js`.
- What it does:
 - validates email, password (8+ characters, a letter and a number), role, and name
 - rejects duplicate emails
 - hashes the password with `bcrypt`
 - creates the user and profile in one transaction
 - returns a token

Login and session (#22)

- Own `POST /api/auth/login`, `GET /api/auth/me`, and `middleware/auth.js`.
- Covers JWT signing with a 2-hour expiry, and checking the `Authorization: Bearer` header on protected routes.
- A wrong password and an unknown email return the same error on purpose, so attackers can't tell which emails exist.

Tests

- Own `tests/auth.test.js` and `tests/helpers.js`.
- Run `npm test`, confirm everything passes, and add at least one test of your own. An extra edge case shows real contribution beyond the generated code.

Also: set up `server/.env` locally and make sure `.env` is never committed.

Member C: Backend, profile and resumes (Philip)

Profile API (#24)

- Own `routes/profile.js`: GET `/api/profile` and PUT `/api/profile`.
- It validates phone, LinkedIn URL, non-empty name, bio length (max 1000), and that skills is a list.
- Skills are stored as JSON.
- A PUT only updates the fields that were sent, so a partial update doesn't wipe the other fields.

Resume upload (#26)

- Own the `resumes` table and POST `/api/resumes`.
- It uses Multer to enforce:
 - PDF or DOCX only, and the file extension must match the type
 - 5 MB maximum
 - 5 resumes per user
 - recruiters are blocked
- Files are saved under random names so users can't overwrite each other's files, and the first upload becomes the primary resume.

Resume management (#28)

- Own the list, download, set-primary, and delete endpoints.
- Every query checks that the resume belongs to the logged-in user. Another user's resume returns 404.
- Deleting the primary resume promotes the newest remaining one.

Tests and docs

- Own `tests/profile.test.js` and `tests/resumes.test.js`.
 - Keep the API table in the README accurate.
 - Be ready to explain the security checks (file type, size, ownership); TAs often ask about upload safety.
-

Member D: Frontend, authentication and demo (Daryl)

Registration page (#21)

- Own `Register.jsx` and `Field.jsx`: the role picker, field-level error messages from the API, and the redirect to the profile page after signup.

Sign-in and session handling (#23)

- Own `Login.jsx`, `AuthContext.jsx`, `api.js`, `App.jsx`, and `AuthLayout.jsx`.
- Covers:
 - storing the token and restoring the session on page reload
 - redirecting logged-out users to sign-in
 - sending logged-in users away from the auth pages
 - signing out

Client setup

- Own `client/package.json`, `vite.config.js`, and `index.html`.
- Confirm `npm run dev` works on at least two teammates' machines, ideally one Windows and one Mac.

Sprint 1 demo (#31)

- Write a 3–5 minute demo script:
 1. Register a job seeker.
 2. Show a validation error.
 3. Sign out and sign back in.
 4. Edit the profile.
 5. Upload two resumes.
 6. Change the primary resume.
 7. Show a rejected file type.
 8. Show that a recruiter account has no Resumes tab.
 - Seed test accounts beforehand and do a dry run before the lab.
-

Member E: Frontend, profile and resumes (Vincent)

Profile page (#25)

- Own `Profile.jsx`: loading and saving, the success and error banners, a comma-separated skills field, and a character counter on the About field.
- Recruiters see a Company field and no Skills field.

Resume upload UI (#27)

- Own the upload area in `Resumes.jsx`: drag and drop plus a file picker, an "Uploading..." state, and showing the API's error message (wrong type, too big, limit reached).

Resume list UI (#29)

- Own the list: name, size, and date for each resume, the "Primary" tab, and the Download, Make primary, and Delete buttons (Delete asks for confirmation).

Layout and styling

- Own `AppLayout.jsx` (top bar and navigation) and `styles.css`.
- Check the pages at phone width and keyboard focus visibility.
- Take screenshots of each screen for the demo and for the team's reference in future sprints.

Member F: Documentation and DevOps (Ben)

README (#12)

- Finalize `README.md`: team table with names, IDs, GitHub handles, and roles, plus the real repo URL.
- Test the setup instructions by following them exactly on a fresh clone, and fix anything that doesn't work.
- Add the original team feature to the features table once the brainstorm decides it.

CI (#15)

- Own `.github/workflows/ci.yml`.
- Confirm it runs on a PR: API tests plus client build. Get the first green run and make it a required check in branch protection (coordinate with A).

Templates and hygiene

- Own the issue and PR templates in `.github/`, `.gitignore`, and `server/.env.example`.
- Make sure no database, upload, or `.env` files end up in the repo.

Submission document (#30)

- Finalize `CareerConnect_Sprint1_Submission.docx`: cover page with all names and IDs, a copy of the final README, and a working repo link.
- Test the link while logged out, or confirm the instructor has access.
- Submit it before the deadline.

User Stories

CareerConnect User Stories

Job Seeker Stories

US-01 — User Registration (Sprint 1)

As a **job seeker**, I want to create an account so that I can access CareerConnect features.

US-02 — User Login (Sprint 1)

As a **registered user**, I want to log in securely so that I can access my personal account and information.

US-03 — Profile Management (Sprint 1)

As a **job seeker**, I want to create and edit my profile so that my personal and professional information remains up to date.

US-04 — Resume Upload

As a **job seeker**, I want to upload my resume so that I can use it when applying for jobs.

US-05 — Resume Management

As a **job seeker**, I want to view, replace, or delete my uploaded resumes so that I can keep my resume information current.

US-06 — Browse Job Postings

As a **job seeker**, I want to browse available job postings so that I can discover employment opportunities.

US-07 — Search for Jobs

As a **job seeker**, I want to search for jobs using keywords so that I can quickly find relevant opportunities.

US-08 — Filter Job Postings

As a **job seeker**, I want to filter job postings based on criteria such as location or job type so that I can narrow down my search.

US-09 — Save Jobs

As a **job seeker**, I want to save interesting job postings so that I can easily return to them later.

US-10 — Submit Job Application

As a **job seeker**, I want to submit an application to a job posting so that I can apply directly through CareerConnect.

US-11 — Track Application Status

As a **job seeker**, I want to track the status of my applications so that I know whether each application is Applied, Interview, Offered, or Rejected.

US-12 — Application History Dashboard

As a **job seeker**, I want to view all of my current and previous applications in one dashboard so that I can easily monitor my job-search progress.

US-13 — Application Deadline Reminders

As a **job seeker**, I want to receive reminders about upcoming application deadlines so that I do not miss important opportunities.

Recruiter Stories

US-14 — Create and Manage Job Postings

As a **recruiter**, I want to create, edit, and remove job postings so that I can manage available opportunities for my organization.

AI Feature

US-15 — AI-Assisted Resume Feedback

As a **job seeker**, I want to receive AI-assisted feedback on my resume so that I can identify areas that may need improvement before applying for jobs.